

VEHICLE DETECTION USING YOLO (OPENCV)

PROJECT STATEMENT:

Detect vehicles using YOLO model and count detected objects.

STEP 1: Import Required Libraries.

Loads required libraries for image processing and deep learning.

```
import cv2  
import numpy as np
```

STEP 2: Load YOLO Model.

Initializes the deep learning model.

```
net = cv2.dnn.readNet("yolov3.weights", "yolov3.cfg")
```

STEP 3: Load COCO Class Labels.

Helps to convert class ID numbers into readable labels.

```
with open("coco.names", "r") as f:  
    classes = [line.strip() for line in f.readlines()]
```

STEP 4: Get Output Layers.

YOLO has multiple output layers.

It Identifies the final layers and prepare them for prediction.

```
layer_names = net.getLayerNames()  
output_layers = [layer_names[i - 1] for i in net.getUnconnectedOutLayers()]
```

STEP 5: Load Input Image.

Reads the image and Extracts image dimensions.

```
image = cv2.imread("traffic vechiles.jpg")  
height, width, channels = image.shape
```

STEP 6: Preprocess Image (Create Blob).

Resize image to 416×416 (YOLO standard size).

Normalize pixel values (0–1) and convert image into blob format.

This prepares image for neural network input.

```
blob = cv2.dnn.blobFromImage(image, 1/255.0, (416, 416), swapRB=True, crop=False)
net.setInput(blob)
```

STEP 7: Forward Pass (Object Detection).

The model Scans image, detects objects and returns predictions.

Each detection contains Width & height, Confidence score and Class probabilities.

```
outputs = net.forward(output_layers)
```

STEP 8: Extract Bounding Boxes.

```
boxes = []
confidences = []
class_ids = []
```

Loop through detections:

```
for output in outputs:
    for detection in output:
```

Process Each Detection:

- Get class with highest probability
- Check confidence > 0.5
- Convert relative coordinates to actual pixel values

Stores:

- Box coordinates
- Confidence values
- Class IDs

```
if confidence > 0.5:
    center_x = int(detection[0] * width)
    center_y = int(detection[1] * height)
    w = int(detection[2] * width)
    h = int(detection[3] * height)

    x = int(center_x - w / 2)
    y = int(center_y - h / 2)

    boxes.append([x, y, w, h])
    confidences.append(float(confidence))
    class_ids.append(class_id)
```

STEP 9: Apply Non-Maximum Suppression (NMS).

Removes duplicate overlapping boxes and keeps best detection.

Improves detection quality.

```
indexes = cv2.dnn.NMSBoxes(boxes, confidences, 0.5, 0.4)
```

STEP 10: Filter Only Vehicle Classes.

Only counts Car, Bus, Truck, Motorbike.

Ignore other objects like Person, Bicycle, Dog, Traffic light.

```
vehicle_classes = ["car", "bus", "truck", "motorbike"]
```

STEP 11: Count Vehicles & Draw Boxes.

```
vehicle_count = 0
```

Loop through valid detections:

```
if len(indexes) > 0:
    for i in indexes.flatten():
        label = classes[class_ids[i]]

        if label in vehicle_classes:
            vehicle_count += 1

            x, y, w, h = boxes[i]
```

Draw rectangle:

```
cv2.rectangle(image, (x, y), (x+w, y+h), (0, 255, 0), 2)
```

Add label:

```
cv2.putText(image, label, (x, y-10),  
            cv2.FONT_HERSHEY_SIMPLEX, 0.6,  
            (0, 255, 0), 2)
```

STEP 12: Display Total Vehicle Count.

```
cv2.putText(image, f"Total Vehicles: {vehicle_count}",  
            (20, 40),  
            cv2.FONT_HERSHEY_SIMPLEX,  
            1,  
            (0, 0, 255),  
            3)
```

Print result:

```
print("Total Vehicles Detected:", vehicle_count)
```

STEP 13: Show Output Window.

Displays final result with green bounding boxes and red total count text.


```
cv2.imshow("Vehicle Detection", image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output:

Total Vehicles Detected: 26

